Introduction

Introduction I

Editorial review is one of the longest-lived bottlenecks in scientific writing — and an obvious target for the kind of *reproducible computational workflow* advocated by [@peng2011reproducible]. A working draft accumulates structural drift (skipped heading levels, sections that have grown to 2,000 words while their siblings sit at 200), readability drift (a once-clean introduction now reads at the Gunning Fog level of a legal contract [@gunning1952technique]), and citation drift (`[@key]` references that no longer exist in `references.bib`, or bibliography entries that nobody actually cites). The traditional remedy — a senior co-author re-reading the manuscript with [@strunk2000elements] in hand — is slow, non-reproducible, and impossible to apply continuously.

Introduction II

`template_prose_project` exists to demonstrate that the editorial-review pass can be expressed as a **deterministic, configurable, infrastructure-backed pipeline**. Like the permanent sibling exemplar `template_code_project` (numerical research) and the optional search add-on `template_search_project` (literature discovery), this project carries no novel research contribution of its own; its purpose is to show how to compose existing template infrastructure into a complete, reproducible research workflow — in this case, an editorial workflow.

Introduction III

The architecture is simple. `manuscript/config.yaml` defines policy: target grade-level band, citation-density floor, heading-structure rules, bibliography-consistency policy. `src/pipeline.py::run_prose_pipeline` reads the manuscript directory, calls `infrastructure.prose.analyze_manuscript` to produce a `ManuscriptReport`, cross-checks the cited keys against `infrastructure.reference.citation.parse_bibfile` for the `references.bib`, evaluates each configured check, and writes a markdown review report alongside JSON artefacts and three figures. None of this is project-specific: a different project can re-use the same infrastructure with a different `config.yaml` and a different manuscript directory.

The remainder of this paper documents the methodology ([@sec:methodology]), the run-time results on the bundled manuscript ([@sec:results]), and the architectural lessons drawn from doing the prose work alongside the code and literature exemplars in the same template ([@sec:conclusion]).